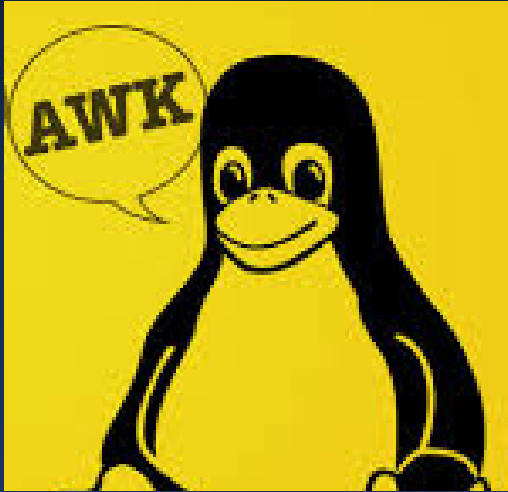




Higher Diploma in Computer Science

Computer Systems & Networks





```
$ awk 'pattern { action }'
```

awk

awk

```
awk -F":" ' /linux/ {print $1}'
```

- originally designed for pattern searching and text manipulation
- Basic function:
 - search files for lines that contain certain patterns
 - on match, perform some action(s)
- excellent tool for processing files with rows and column
- has string manipulation functions

When could you use awk?

You can:

- print only the columns you want
- print them in the order you want
- format the output i.e. an empty line between each line
`$cat employee.txt | awk '{print $0 "\n"}'`
- format with **printf**
- use conditions to print only lines/columns you want

awk

`awk -F":" '/linux/ {print $1}'`

pattern

Field Separator

Action

```
$ awk 'pattern { action }'
```

The pattern comes first, and then the action

- Action statements are enclosed in { and }.

Either the pattern may be missing, or the action may be missing, but, of course, not both.

```
$ awk '/caroline/ {print $1,$2}' employee.txt
```

- If the pattern is missing, the action is executed for every single record of input.
- A missing action is equivalent to { print } which prints the entire record by default.

Categories of awk

There are three major versions of awk.

1. The traditional Unix awk and
2. the newer Gawk. Gawk is the GNU Project's implementation of the awk programming language.
 - Gawk has several extensions to the original awk
3. The **New awk** Nawk.

`$which awk #check version`

\$ awk 'pattern { action }'

AWK is a line-oriented language

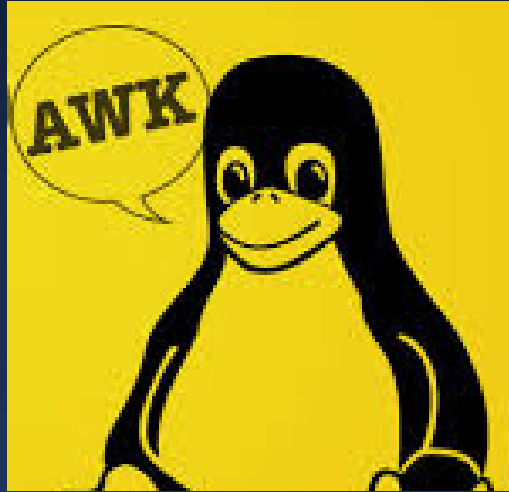
- By default awk prints every line of data from the specified file
 - awk treats tab or whitespace for file separator by default.

\$ awk '{print}' employee.txt

\$ awk '{print \$2}' employee.txt

\$ awk '/C/ {print \$2}' employ*

```
compsys@compsys-virtualbox:~$ awk '/c/{print}' employee.txt
200 caroline developer technology €5500
300 mary sysadmin technology €7000
500 eamonn dba technology €6000
compsys@compsys-virtualbox:~$ awk '/C/{print}' employee.txt
compsys@compsys-virtualbox:~$
```



awk
variables

for column processing

Manipulate and extract individual fields

- divides a file into lines called *records*
- automatically splits each record into *fields*
- *fields* default delimiter is the space character
 - Can change using -F optional parameter followed by the desired separator.

```
$ awk 'BEGIN {FS=":"}{print $1}' /etc/passwd
```

- The *fields* are accessed by special variables:
 - the first field is \$1, the second \$2, and so on.
 - The special variable \$0 contains the entire record before splitting.

awk to change the field separator

- Two actions:

```
awk 'BEGIN{FS=":"; OFS="-"} {print $1,$6,$7}' /etc/passwd
```

1. Find the colons
2. Use : as the separator but on output, print the dash – as the field separator

By default awk doesn't print out the separators

Manipulate and extract individual fields

- awk can process text files.
- assigns variables for each data field found

\$0 for the whole line.

\$1 for the first field.

\$2 for the second field.

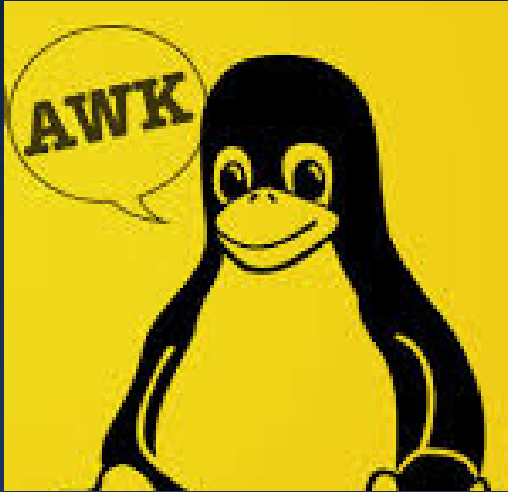
\$n for the nth field.

- a. Print out only the first field of the **employee.txt** file
- b. Print out only the first and third fields of the **employee.txt** file

Manipulate and extract individual fields

- Built-in variables have values already defined in Awk

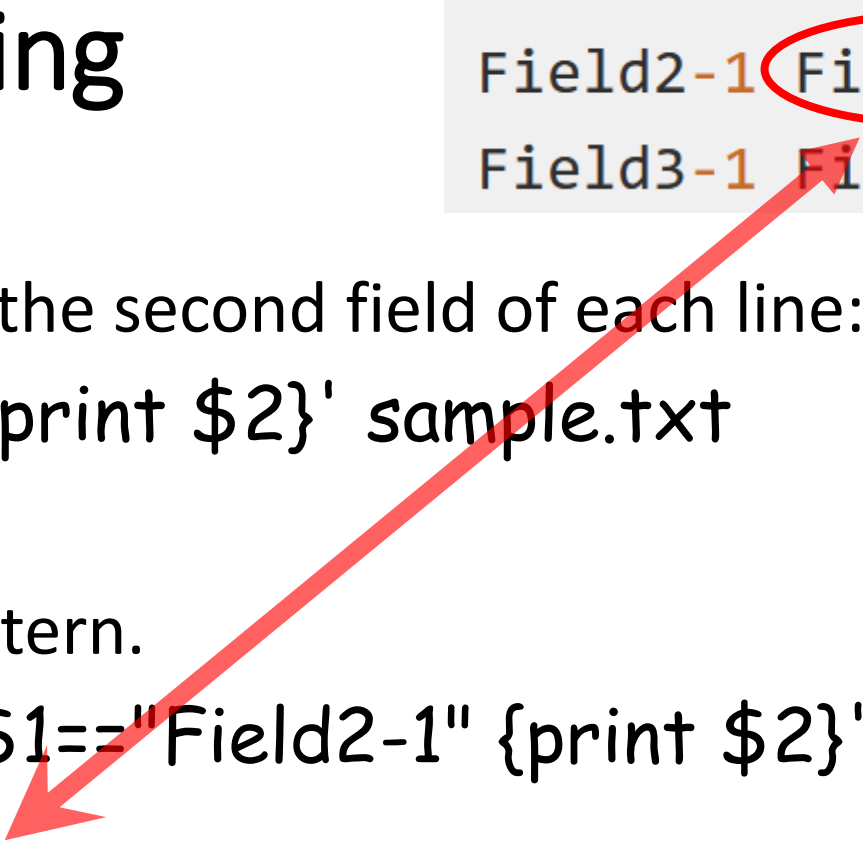
<u>Variable name</u>	<u>Description</u>
FS	field separator (default space)
NF	No. of Fields ; prints last field in the current record
NR	current record/line number=total count of processed records
\$0	whole line
\$n	n-th field
FNR	current record number in the current file



awk

for pattern matching

Pattern matching



Field1-1	Field1-2	Field1-3
Field2-1	Field2-2	Field2-3
Field3-1	Field3-2	Field3-3

To use **awk** to extract the second field of each line:

```
$ awk '{print $2}' sample.txt
```

You can also add a pattern.

```
$ awk '$1=="Field2-1" {print $2}' sample.txt
```

Field2-2

second field will only be printed if the first field is equal to *Field2-1*

Pattern Match fields

- Provide a regular expression enclosed by slash / characters to match records:

```
-$ awk '/caroline/ {print $1,$2}' employee.txt
```

- You can also use fields and a comparison expression as pattern matching criteria:

```
-$ awk '$1==600 {print $1,$2}' employee.txt
```

awk Comparison & Logical Operators

awk has all the usual:

1) comparison operators

and 2) arithmetic operators

< less than

<= less than or equal to

== equal to

>= greater than or equal to

!= not equal to

> greater than

+ add

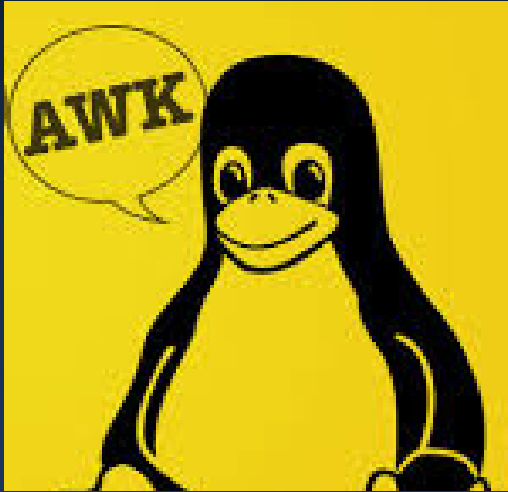
- subtract

* mult

/ divide

% modulo

<space> concatenate



awk
printing

for pattern matching

Printing specific output

```
compsys@compsys-virtualbox:~/temp/tutorial$ ls -l
total 8
drwxrwxr-x 4 compsys compsys 4096 Aug 31 13:49 Dec
drwxrwxr-x 2 compsys compsys 4096 Aug 31 13:02 Nov
```

- We want to extract only the owner, size, and file name:

<code>\$ls -l awk '{print \$3 \$5 \$9}'</code>	# poor output
<code>\$ls -l awk '{print \$9 " " " \$5 " "\$3}'</code>	#add spaces manually
<code>\$ls -l awk '{print \$5"\t"\$1}'</code>	# force a tab \t
<code>\$ls -l awk -v OFS='\t' '{print \$9, \$5, \$3}'</code>	#use Output Field Separator tab

Printing user friendly specific output

```
compsys@compsys-virtualbox:~$ awk '/manager/ {print "The " $4 " manager name is: " $2 " who claimed expenses of: " $(NF)}' employee.txt
The sales manager name is: frank who claimed expenses of: €5000
The marketing manager name is: rosanne who claimed expenses of: €9500
compsys@compsys-virtualbox:~$ █
```

```
compsys@compsys-virtualbox:~$ awk '/[Mm]anager/ {print "The manager " $2 " claimed expenses of " $(NF)}' employee.txt
The manager frank claimed expenses of €5000
The manager rosanne claimed expenses of €9500
The manager Caroline claimed expenses of €1234
compsys@compsys-virtualbox:~$ cat empl*
300 frank manager sales €5000
400 caroline developer technology €5500
999 catherine dev tech €4500
100 mary sysadmin technology €7000
200 rosanne Manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
echo "There are no $ currency, only € in employee.txt"
just another example katherine
666 Katherine dev tech €6000
123 Caroline Manager Sales €1234
```

Print fields using printf

- After splitting, awk assigns each field to a numbered variable
 - the first field is \$1, the second \$2, and so on.
 - The special variable \$0 contains the entire record before splitting.

```
compsys@compsys-virtualbox:~$ awk '{printf("%-40s %s\n", $1,$2)}' employee.txt
300      frank
400      caroline
999      catherine
100      mary
200      rosanne
500      eamonn
600      colm
echo     "There
just     another
666      Katherine
123      Caroline
```

Modifiers for **printf** Formats

The **printf** command in awk allows you to print formatted output using format specifiers

<u>SYNTAX:</u>	<u>%[modifier]control-letter</u>
c	Prints numeric output as a string.
d	Prints an integer value.
f	Prints float values.
o	Prints an octal value.
s	Prints a text string.
e	Prints scientific numbers.

Modifiers for **printf** Formats

SYNTAX: %[modifier]control-letter

```
compsys@compsys-virtualbox:~$ awk 'BEGIN{  
> x = 100 * 50  
> printf "The result is %e\n",x  
> }'  
The result is 5.000000e+03
```

e Prints scientific numbers.

Modifiers for **printf** Formats

```
printf "%4s", "computersystems"
```

- Inserting any number after the '%' sign forces the field to be expanded to this width (*at least*)
 - default is to pad with spaces on the left,
 - "-" causes padding on the right

```
awk { printf "%-10s %s\n", $1, $2 }' employee.txt
```

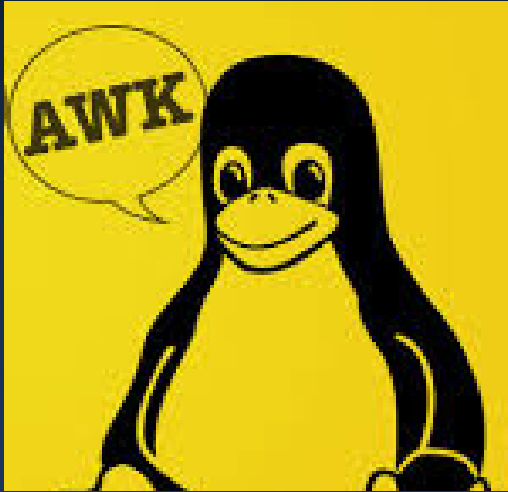
Extract multiple fields from record

- You can also print multiple fields by separating them with commas.
EG print column one and three, use this command:

```
$ awk '{ print $1, $3 }' employee.txt
```

- You could use the `printf` function and provide a 40 character padding to the right of first columns:

```
$ awk '{ printf("%-40s %s\n", $1, $3) }' filename.txt
```

BEGIN,
main and
END

syntax

BEGIN and END

- A BEGIN rule is executed once only, before the first input record is read.
- Likewise, an END rule is executed once only, after all the input is read

```
GNU nano 6.2  
awk '  
BEGIN { print "Analysis of \"caroline\"" }  
/caroline/ { ++n }  
END {  
    print "\"Caroline\" appeared in ", n, "records."  
}' employee.txt
```

What's the output of:

```
awk 'BEGIN { for(i=1; i<=10; i++) print "The square root of", i, "is", i*i;}'
```